Actividades
Práctico-Experimentales Nro. 003

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	03
Título de la Práctica	Mapeo de referencias en memoria y la auditoría de fugas de memoria.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Integrantes del Grupo	Danny Beltran Isaac Vire Noelia Bustan Lucas Orellana Axel Jimenez
Fecha	Jueves 15 y Viernes 16 de octubre 2026
Horario	Jueves 15 de 08:30 – 10:30 Viernes 16 de 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Silabo	3 horas

2. Objetivo(s) de la Práctica:

- Modelar el ciclo de vida de los objetos en memoria, diferenciando físicamente la asignación estática (Stack) de la asignación dinámica (Heap) dentro del motor V8 de JavaScript.
- Auditar y cuantificar empíricamente el desperdicio de memoria (fugas o Memory Leaks) originado por referencias colgantes y cierres léxicos mal gestionados.
- Aplicar principios de Green Computing induciendo manualmente el Recolector de Basura (Garbage Collector) para medir el ahorro de memoria RAM y el decrecimiento de la complejidad espacial $S(n)$ a lo largo del tiempo.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para investigación en documentación oficial de V8.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El servidor satelital desarrollado en la Práctica 001 lleva funcionando 48 horas continuas procesando coordenadas, pero repentinamente colapsa mostrando el error FATAL ERROR: Ineffective mark-compacts near heap limit Allocation failed - JavaScript heap out of memory. El equipo debe auditar el código, encontrar por qué la memoria crece a un ritmo de $O(n)$ sin límite, y aplicar una solución arquitectónica para estabilizar el consumo.
- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo opera el algoritmo *Mark-and-Sweep* (Marcado y Barrido) del *Garbage Collector* (GC) y por qué las referencias globales impiden su ejecución.
- **Modalidad:** Individual.

Tarea 1: Simulación de una Fuga de Memoria (Memory Leak)

Para poder auditar un problema de memoria, primero debemos crearlo modelando un historial de ubicaciones satelitales que jamás se descarta.

1. En el directorio de su proyecto, cree un archivo llamado lab03_memory_leak.js.
2. Implemente el siguiente código que simula la llegada asíncrona de datos que se acumulan en el entorno global (Heap).

```
const { performance } = require('perf_hooks');

// Referencia global: Actúa como raíz (root), el GC no puede eliminar lo que esté aquí.
const historialSatelitalGlobal = [];

console.log("[Auditoría] Iniciando simulación de Memory Leak...");

function procesarCoordenada() {
    // Simulamos un paquete de datos entrante (aprox. varios kilobytes)
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        // Carga útil pesada para acelerar el llenado de la memoria
        payload: new Array(100000).fill('datos_topograficos_basura')
```

```
    };
   // El error lógico: Almacenamos todo indefinidamente causando S(n) = O(n)
    historialSatelitalGlobal.push(paquete);
}

// Monitoreo de la memoria Heap cada 100 iteraciones
for (let i = 1; i <= 500; i++) {
    procesarCoordenada();
   if (i % 100 === 0) {
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM consumida: ${memoryUsed} MB`);
    }
}
console.log(`[Auditoría] Finalizado. El arreglo retiene ${historialSatelitalGlobal.length} registros en el Heap.`);
```

3. Ejecute el script con node lab03_memory_leak.js y observe cómo la RAM crece linealmente.

Tarea 2: Forzado del Recolector de Basura (Garbage Collector)

Por defecto, el GC de Node.js es no determinista. Para medir si estamos solucionando la fuga, forzaremos su ejecución.

1. Duplique el archivo anterior y nómbrelo lab03_gc_force.js.
2. Modifique el ciclo for para invocar al Recolector de Basura. Node.js requiere que esto se haga mediante una bandera especial de ejecución.

```
// Añada esto dentro de su ciclo iterativo, después del push al historial
if (i % 100 === 0) {
    // Si global.gc está expuesto, lo forzamos a limpiar
    if (global.gc) {
        global.gc(); // Ejecutamos el barrido manual
    }
    const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
    console.log(`Iteración ${i} | RAM post-GC: ${memoryUsed} MB`);
}
```

3. **Ejecución estricta:** Para exponer la función gc(), ejecute en terminal: node --expose-gc lab03_gc_force.js.

4. **Análisis:** Notará que la RAM sigue creciendo. El *Garbage Collector* no eliminó la basura porque el arreglo global historialSatelitalGlobal mantiene una **referencia activa** hacia los objetos. El algoritmo *Mark-and-Sweep* los marca como "en uso".

Tarea 3: Resolución y Mantenimiento Ecológico (O(1) Espacial)

1. Cree el archivo lab03_solucion.js.
2. Refactorice la lógica eliminando referencias obsoletas, permitiendo que el GC las reclame, manteniendo un historial cíclico de máximo 50 registros espaciales.

```
const historialSatelital = [];
 const LIMITE_HISTORIAL = 50; // Constante C, por lo que S(n) = O(1)

 function procesarCoordenadaOptimizada() {
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        payload: new Array(100000).fill('datos_topograficos_optimizados')
    };

    historialSatelital.push(paquete);

    // Desvinculación léxica: Eliminamos la referencia al objeto más antiguo
    if (historialSatelital.length > LIMITE_HISTORIAL) {
        historialSatelital.shift(); // Quitamos el primer elemento (FIFO)
    }
}

 for (let i = 1; i <= 1000; i++) {
    procesarCoordenadaOptimizada();

    if (i % 200 === 0) {
        if (global.gc) global.gc(); // Forzamos el GC
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM Controlada: ${memoryUsed} MB`);
    }
}
```

3. Ejecute: node --expose-gc lab03_solucion.js. Compruebe cómo el consumo de memoria alcanza un techo y se mantiene estable, resolviendo el colapso del servidor.

6. Resultados Obtenidos:

Resultados de la Tarea 1:

```js
// Se simula la llegada masiva de datos satelitales que se acumulan sin límite
// en historialSatelitalGlobal. Es una variable global, por lo que actúa como raíz.
// Entonces el Garbage Collector no libera lo que sigue siendo alcanzable desde ella.
// Como resultado la memoria crece con cada paquete, S(n) = O(n). Por último,
// Se monitorea el Heap para saber cuanto crece la memoria cada 100 paquetes.

const { performance } = require('perf_hooks');

const historialSatelitalGlobal = [];

console.log("[Auditoría] Iniciando simulación de Memory Leak...");

function procesarCoordenada() {
    // Simulamos un paquete de datos entrante (aprox. varios kilobytes)
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        // Carga útil pesada para acelerar el llenado de la memoria
        payload: new Array(100000).fill('datos_topograficos_basura')
    };
   // El error lógico: Almacenamos todo indefinidamente causando S(n) = O(n)
    historialSatelitalGlobal.push(paquete);
}

// Monitoreo de la memoria Heap cada 100 iteraciones
for (let i = 1; i <= 500; i++) {
    procesarCoordenada();
   if (i % 100 === 0) {
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM consumida: ${memoryUsed} MB`);
    }
}
console.log(`[Auditoría] Finalizado. El arreglo retiene ${historialSatelitalGlobal.length} registros en el Heap.`);
```

```
[Auditoría] Iniciando simulación de Memory Leak...
Iteración 100 | RAM consumida: 81 MB
Iteración 200 | RAM consumida: 157 MB
Iteración 300 | RAM consumida: 233 MB
Iteración 400 | RAM consumida: 310 MB
Iteración 500 | RAM consumida: 386 MB
[Auditoría] Finalizado. El arreglo retiene 500 registros en el Heap.
```

Resultados de la Tarea 2:

```js
// Simula la llegada masiva de datos satelitales que se acumulan sin límite
// en historialSatelitalGlobal. Cada 100 paquetes se fuerza el Garbage Collector
// (que se ejecuta con --expose-gc), pero no libera memoria, ya que el arreglo es
// global y sigue siendo alcanzable desde la raíz, por lo que el GC marca todos
// los paquetes como si estuvieran en uso y no puede eliminarlos.
// Como resultado la memoria sigue creciendo con cada paquete, S(n) = O(n).
const { performance } = require('perf_hooks');

const historialSatelitalGlobal = [];
console.log("[Auditoría] Iniciando simulación de Memory Leak...");

function procesarCoordenada() {
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        payload: new Array(100000).fill('datos_topograficos_basura')
    };
    historialSatelitalGlobal.push(paquete);
}

for (let i = 1; i <= 500; i++) {
    procesarCoordenada();

    if (i % 100 === 0) {
        // Si global.gc está expuesto, lo forzamos a limpiar
        if (global.gc) {
            global.gc(); // Ejecutamos el barrido manual
        }
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM post-GC: ${memoryUsed} MB`);

    }
}
console.log(`[Auditoría] Finalizado. El arreglo retiene ${historialSatelitalGlobal.length} registros en el Heap.`);
```

```
[Auditoría] Iniciando simulación de Memory Leak...
Iteración 100 | RAM post-GC: 81 MB
Iteración 200 | RAM post-GC: 157 MB
Iteración 300 | RAM post-GC: 233 MB
Iteración 400 | RAM post-GC: 310 MB
Iteración 500 | RAM post-GC: 386 MB
[Auditoría] Finalizado. El arreglo retiene 500 registros en el Heap.
```

Resultados de la Tarea 3:

```javascript
// Limita el historial a un máximo de 50 paquetes con LIMITE_HISTORIAL.
// Cuando se supera ese tope, shift() elimina el paquete más antiguo (se usa FIFO)
// y con ello su referencia, entonces el Garbage Collector puede limpiarlo de la memoria.
// Como resultado el consumo se estabiliza en una constante, S(n) = O(1),
// lo que resuelve la fuga de memoria de las tareas anteriores.
const historialSatelital = [];
const LIMITE_HISTORIAL = 50; // Constante C, por lo que S(n) = O(1)

function procesarCoordenadaOptimizada() {
    const paquete = {
        timestamp: Date.now(),
        lat: Math.random() * 90,
        lng: Math.random() * 180,
        payload: new Array(100000).fill('datos_topograficos_optimizados')
    };

    historialSatelital.push(paquete);

    // Desvinculación léxica: Eliminamos la referencia al objeto más antiguo
    if (historialSatelital.length > LIMITE_HISTORIAL) {
        historialSatelital.shift(); // Quitamos el primer elemento (FIFO)
    }
}

for (let i = 1; i <= 1000; i++) {
    procesarCoordenadaOptimizada();

    if (i % 200 === 0) {
        if (global.gc) global.gc(); // Forzamos el GC
        const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
        console.log(`Iteración ${i} | RAM Controlada: ${memoryUsed} MB`);
    }
}
```

```
Iteración 200 | RAM Controlada: 42 MB
Iteración 400 | RAM Controlada: 42 MB
Iteración 600 | RAM Controlada: 42 MB
Iteración 800 | RAM Controlada: 42 MB
Iteración 1000 | RAM Controlada: 42 MB
```

Análisis de los resultados de las Tareas

Tarea	1	2	3
Crecimiento de Ram	De manera lineal	De manera lineal	Crece al inicio y se estabiliza
Complejidad espacial	S(n) = O(n)	S(n) = O(n)	S(n) = O(1)

¿El Garbage Collector libera memoria?	No lo puede hacer	No puede, aunque se lo forzara	Puede liberar memoria conforme se alcance el límite de paquetes

La diferencia entre las tareas no depende del Garbage Collector, sino del código. En las Tareas 1 y 2, el arreglo global conservaba una referencia a todos los paquetes, por lo que seguían siendo alcanzables desde la raíz, el GC los marcaba como si estuvieran en uso y no liberaba nada, incluso al forzarlo. En la Tarea 3 se limita el historial a 50 paquetes, y cuando se supera ese límite, shift() elimina el más antiguo (FIFO) y con él su referencia. Por lo que, al volverse inalcanzable, el GC lo reclama. Así, la memoria llega a un techo constante, S(n) = O(1), y el programa puede funcionar sin inconvenientes de memoria.

7. Preguntas de Control:

Desde una perspectiva filosófica profunda y analítica, responda en su informe:

- **Causa Material: Al declarar un objeto complejo dentro de una función, ¿en qué "lugar material" (Stack o Heap) se asienta su estructura principal, y en qué lugar se asienta el puntero que lo referencia?**

Al declarar un objeto complejo dentro de una función, como el caso de paquete en procesarCoordenada(), su estructura principal se asienta en el Heap, donde V8 también almacena el Map que define su forma y propiedades. El puntero que lo referencia, es decir, la variable local paquete en este caso, se asienta dentro del Stack, en lo que sería el marco de ejecución de la función. Cuando la función termina, su marco se destruye y el puntero desaparece, mientras que el objeto permanece en el Heap mientras exista otra referencia que lo alcance. Por esto, cuando historialSatelitalGlobal.push(paquete) crea una referencia desde el ámbito global, el objeto sigue siendo alcanzable y el GC no puede liberarlo.

- **Causa Formal: El contexto global de JavaScript tiene una "forma" que abarca todo el ciclo de vida del programa. ¿De qué manera la estructura formal del alcance (*scope*) global actúa como una jaula que le prohíbe actuar al Recolector de Basura?**

La estructura formal del alcance global actúa como una jaula para el Recolector de Basura porque, dentro del algoritmo de marcado y barrido, el objeto y entorno global están definidos intrínsecamente como una Raíz de Alcanzabilidad permanente.
A diferencia de las funciones locales cuyos contextos de ejecución se destruyen al salir de la pila, el contexto global persiste ininterrumpidamente durante todo el ciclo de vida de la aplicación. Esto significa que cualquier variable, función o closure colgada del alcance global crea una arista o camino directo en el grafo de memoria hacia esa raíz indestructible; y dado que la regla fundamental del

motor de JavaScript prohíbe taxativamente liberar cualquier nodo alcanzable desde una raíz, la permanencia del ámbito global atrapa a todos sus elementos vinculados en un estado de supervivencia perpetua, impidiendo que el Recolector de Basura pueda actuar a menos que la ruta de referencias se rompa explícitamente reasignando el valor a null o recurriendo a estructuras de referencias débiles como weak map.

- **Causa Eficiente:** El algoritmo de *Mark-and-Sweep* actúa como un destructor activo. Sin embargo, en un *Memory Leak*, ¿quién o qué es el agente responsable (causa eficiente) que detiene la hoja de la guadaña del Recolector de Basura?

El motivo eficiente de un Memory Leak es el código del programa que tiene referencias innecesarias a objetos que ya no se utilizan. Estas referencias hacen imposible que el recolector de basura (Garbage Collector), mediante el algoritmo Mark-and-Sweep, identifique dichos objetos como inalcanzables y libere la memoria que ocupan. Esto puede ocurrir por variables globales, temporizadores, eventos o estructuras de datos que conservan referencias activas. Por tanto, el problema no está en el recolector de basura, sino en la forma en que el programa administra las referencias a los objetos .

- **Causa Final: Si el hardware permite almacenar Gigabytes de información temporal, ¿cuál es el propósito final (telos) de programar una destrucción intencional de los datos que nosotros mismos hemos creado con tanto esfuerzo computacional?**

El propósito final de la destrucción o liberación intencional de datos no es desperdiciar el esfuerzo computacional, sino que es garantizar la continuidad funcional, la baja latencia y la eficiencia energética del sistema en un entorno de recursos finitos. Aunque el hardware disponga de Gigabytes el espacio dinámico sigue siendo limitado y requiere liberar constantemente la memoria ocupada para objetos no alcanzables para permitir nuevas asignaciones y prevenir fugas de memoria. Se puede aumentar la perspectiva de la computación verde que es básicamente la eliminación de datos innecesarios reduce la sobrecarga de procesamiento y la transferencia de almacenamiento lo que reduce drásticamente el impacto en el medio ambiente.

- **Esencia y Existencia: Si un paquete de coordenadas queda "olvidado" en el último índice de un arreglo sin que ninguna función lo vuelva a consultar jamás, físicamente ocupa electrones en la RAM, pero lógicamente es inaccesible. ¿Existe este dato en la realidad del software o ha perdido su esencia al perder su utilidad? Argumente.**

Se puede decir que el dato ha perdido su esencia al perder su utilidad , porque dentro del software , la esencia de un dato reside en su significado, propósito y capacidad de interacción, es decir la capacidad de influir en el programa.

El sistema ya no sabe qué significa ese bloque de memoria en términos de negocio o lógica de ejecución. Para el programa, dejó de ser un "paquete de coordenadas" y pasó a ser un conjunto ciego de bytes.

8. Bibliografía

- P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. Available: https://open.palni.org/
- Mozilla Developer Network, "Memory management and Garbage Collection in JavaScript", *MDN Web Docs*, 2023. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management
- Node.js Foundation, "V8 JavaScript engine and CLI options", *Node.js API*, 2023. Available: https://nodejs.org/docs/latest-v20.x/api/cli.html
- C. Bruni, “Fast properties in V8,” *V8 Blog*, Sep. 2017. [Online]. Available: https://v8.dev/blog/fast-properties

9. Anexos

Enlace a GitHub: https://github.com/B3kk4r/Estructura-de-datos-Grupo9.git